

MACHINE LEARNING IN FINANCE

MODULE 1 | LESSON 2

SUPERVISED MODELS: REGRESSION AND HYPERPARAMETERS

Reading Time 60 minutes

|Prior Knowledge | Linear regression

|Keywords | Penalized regressions, linear models, hyperparameters |

In this lesson, we will introduce in a linear regression framework the main techniques for parameter regularization, cross-validation, and hyperparameter tuning.

1. Linear Models and Overfitting

In the linear regression context, a good way to reduce the overfitting of a model is to regularize (constrain) the parameters, sometimes also called "weights." Regularization reduces the degrees of freedom of the model to fit the training data.

In general form, let $J(\theta)$ denote the cost function, during training, of a model with vector of parameters θ . Then, regularization is achieved by adding a penalty term to the MSE of the model, so that the cost function from linear regression becomes:
$$J(\theta) = \text{MSE}(\theta) + \text{Penalty}(\theta)$$

We next describe three methods--Ridge, Lasso, and Elastic Net--which specify penalty, or regularization, terms to constrain the weights. One important aspect is that, in any method, the penalty term should only be added to the cost function during training. In the validation or test samples, we are only interested in evaluating the unregularized performance of the model.

1.1 Ridge

The Ridge method is called a *shrinkage* regularization technique that aims at keeping the smallest values possible for the parameters of the model. To achieve this, the regularization term is of the form
$$\alpha \sum_{i=1}^n \theta_i^2$$
 The hyperparameter α determines how much we wish to regularize the model. If $\alpha=0$, we are trivially not regularizing the model while, if α is very large, then all parameters will be close to zero and our fit of the data will

be a flat line through the average of the labels y in the training sample (Notice that the regularization term above does not include the bias term θ_0).

The cost function for a linear regression with a ridge penalty (Ridge Regression) is then given by
$$J(\theta) = \text{MSE}(\theta) + \frac{1}{2}\alpha \sum_{i=1}^n \theta_i^2$$

Thus, if we perform gradient descent, the update of the parameters will be given by:
$$\theta^{\{\text{next step}\}} = \theta^{\{\text{previous step}\}} - \eta \nabla_{\theta} \text{MSE}(\theta^{\{\text{previous step}\}}) - \alpha \odot \theta^{\{\text{previous step}\}}$$

where α is a column vector of n elements all of which have the value α , except the first term that has a value of 0, in association with the term θ_0 not being regularized, and $\odot$ refers to element-by-element multiplication. Finally, notice that the $1/2$ multiplying α and the parameters in the penalty function is just to get a cleaner derivative once we compute the gradient.

1.2 Lasso

The Lasso (Least Absolute Shrinkage and Selection Operator) penalty is another regularization method that tends to eliminate the weights of the least important features, setting them to zero. The regularization term is of the form:
$$\alpha \sum_{i=1}^n |\theta_i|$$

Notice that this term is an ℓ_1 norm, while the Ridge version uses an ℓ_2 norm of the parameter vector. The hyperparameter α again determines the overall importance of the regularization of parameters. The Lasso Regression cost function then reads as:
$$J(\theta) = \text{MSE}(\theta) + \alpha \sum_{i=1}^n |\theta_i|$$

The Lasso Regression performs a feature selection and outputs a sparse model with few non-zero parameters. Because the Lasso term is not differentiable at $\theta_i=0$, the gradient descent algorithm needs to be adjusted properly using a *subgradient* method:
$$\theta^{\{\text{next step}\}} = \theta^{\{\text{previous step}\}} - \eta \nabla_{\theta} \text{MSE}(\theta^{\{\text{previous step}\}}) - \alpha \odot \begin{pmatrix} 0 \\ \text{sign}(\theta_1) \\ \vdots \\ \text{sign}(\theta_n) \end{pmatrix}$$

where
$$\text{sign}(\theta_i) = \begin{cases} -1 & \text{if } \theta_i < 0 \\ 0 & \text{if } \theta_i = 0 \\ +1 & \text{if } \theta_i > 0 \end{cases}$$

1.3 Elastic Net

The Elastic Net combines Ridge and Lasso regularization in a single optimization, where a hyperparameter determines the relative combination of shrinkage and feature selection when training the model. The Elastic Net cost function is given by:
$$J(\theta) = \text{MSE}(\theta) + r\alpha \sum_{i=1}^n |\theta_i| + \frac{1}{2}\alpha \sum_{i=1}^n \theta_i^2$$

The hyperparameter α determines the overall importance of the regularization of parameters, while the hyperparameter λ balances the mix between feature selection and parameter shrinkage.

Which of the three methods is preferable? Ridge is a good default, but we guess that only a few features are actually useful. You should opt for Lasso or Elastic Net because they remove the features that are useless. In general, a plain Lasso may behave erratically when the number of features is greater than the number of training instances or when several features are strongly correlated. Thus, the Elastic Net is a good choice.

Notice that in an Elastic Net Regression with Gradient Descent optimization, we would have, at least, three hyperparameters: The learning rate η , the degree of overall regularization of the parameters α , and the relative weight of Lasso vs. Ridge regularization λ . Next, we devise methods to obtain "good" values of those hyperparameters.

1.4 Elastic Net Regression and Times Series Momentum

To illustrate the workings of the Elastic Net Regression, we are going to return to our task of predicting stock returns based on past information.

We first upload the necessary packages and define the input feature matrix X and labels y as in the previous lesson. We also divide the sample into a training set and a test set (we will introduce the validation set below).

```
In [ ]: import numpy as np
import pandas as pd
import yfinance as yf

# Getting historical market data from SPY (ETF) (SPY)
df = yf.download("SPY", start="2000-01-01", end="2022-01-01")

In [ ]: df["Ret"] = df["Adj Close"].pct_change()

name = "Ret"
df["Ret10_i"] = (
    df[name].rolling(10).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 10) - 1)
)
df["Ret25_i"] = (
    df[name].rolling(25).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 25) - 1)
)
df["Ret60_i"] = (
    df[name].rolling(60).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 60) - 1)
)
df["Ret120_i"] = (
    df[name].rolling(120).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 120) - 1)
)
df["Ret240_i"] = (
    df[name].rolling(240).apply(lambda x: 100 * ((np.prod(1 + x)) ** (1 / 240) - 1)
)

del df["Open"]
del df["Close"]
```

```
del df["High"]
del df["Low"]
del df["Volume"]
del df["Adj Close"]

df = df.dropna()
df.tail(10)
```

```
In [ ]: df["Ret25"] = df["Ret25_i"].shift(-25)
df = df.dropna()
df.tail(10)
```

```
In [ ]: X, y = df.iloc[:, 0:-1], df.iloc[:, -1]
print(X.shape, y.shape)
```

```
In [ ]: from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=int(len(y) * 0.5), shuffle=False
)
print(X_train.shape, X_test.shape, y_train.shape, y_test.shape)
```

We first study separately the performance of ridge regression and lasso regression on their own. Recall that ridge regression forces the learning algorithm to not only fit the data but also keep the model parameters θ as small as possible.

In the code below, we keep the regularization parameter α fixed to 0.7.

```
In [ ]: import matplotlib.pyplot as plt

# import ridge regression from sklearn library
from sklearn.linear_model import Ridge
```

```
In [ ]: ridgeR = Ridge(alpha=0.7)
ridgeR.fit(X_train, y_train)
y_pred_ridge = ridgeR.predict(X_train)
y_pred_test_ridge = ridgeR.predict(X_test)

# calculate mean squared error
mean_squared_error = np.mean((y_pred_test_ridge - y_test) ** 2)
print("Mean squared error on test set", mean_squared_error)
```

```
In [ ]: # get ridge coefficient and print them
ridge_coefficient = pd.DataFrame()
ridge_coefficient["Columns"] = X_train.columns
ridge_coefficient["Coefficient Estimate"] = pd.Series(ridgeR.coef_)
print(ridge_coefficient)
```

```
In [ ]: # plotting the coefficient score
fig, ax = plt.subplots(figsize=(15, 7))

color = [
    "tab:gray",
    "tab:blue",
    "tab:orange",
    "tab:green",
    "tab:red",
```

```

        "tab:purple",
        "tab:brown",
        "tab:pink",
        "tab:gray",
        "tab:olive",
        "tab:cyan",
        "tab:orange",
        "tab:green",
        "tab:blue",
        "tab:olive",
    ]

ax.bar(
    ridge_coefficient["Columns"], ridge_coefficient["Coefficient Estimate"], col
)

plt.style.use("ggplot")
plt.show()

```

Check how the model parameters have changed from training the model with the ridge regularization relative to the simple linear regression model. The parameter with the highest absolute value in the trained linear regression, the one associated with the most recent daily return, is adjusted downwards (in absolute terms). In contrast, the parameters on the most distant lags increase their relative importance. However, ridge regularization seems to add little in terms of prediction ability relative to linear regression (compared with the results we obtained in Lesson 1).

Let's see how we can implement Lasso regularization in our application.

```

In [ ]: # import Lasso regression from sklearn library
        from sklearn.linear_model import Lasso

        # Train the model
        lasso = Lasso(alpha=0.0001)
        lasso.fit(X_train, y_train)
        y_pred_lasso = lasso.predict(X_train)
        y_pred_test_lasso = lasso.predict(X_test)

        # Calculate Mean Squared Error
        mean_squared_error = np.mean((y_pred_test_lasso - y_test) ** 2)
        print("Mean squared error on test set", mean_squared_error)

```

```

In [ ]: lasso_coeff = pd.DataFrame()
        lasso_coeff["Columns"] = X_train.columns
        lasso_coeff["Coefficient Estimate"] = pd.Series(lasso.coef_)

        print(lasso_coeff)

```

Notice how Lasso regularization shrinks further the magnitude of the parameters relative to ridge regression, but none reaches a value of zero. Thus, none of the input features of the model are considered as irrelevant by the algorithm. Still, the trained model does not fare better than the Ridge version.

Let's try now with the Elastic Net Regression model that combines both regularization features. Notice that the parameter "l1_ratio" in the code below determines the relative

importance of lasso against ridge regularization in the model.

We can see from the out-of-sample MSE that the combination of both regularization methods contributes little to our prediction problem, as we would expect from observing their separate performance.

```
In [ ]: # import model
        from sklearn.linear_model import ElasticNet

        # Train the model
        e_net = ElasticNet(alpha=0.0001, l1_ratio=0.1)
        e_net.fit(X_train, y_train)

        # calculate the prediction and mean square error
        y_pred_elastic = e_net.predict(X_test)
        mean_squared_error = np.mean((y_pred_elastic - y_test) ** 2)
        print("Mean Squared Error on test set", mean_squared_error)
```

```
In [ ]: e_net_coeff = pd.DataFrame()
        e_net_coeff["Columns"] = X_train.columns
        e_net_coeff["Coefficient Estimate"] = pd.Series(e_net.coef_)
        e_net_coeff
```

2. Hyperparameter Tuning

Let's assume that you now have a shortlist of promising models. You now need to fine-tune them. Let's look at a few ways you can do that.

2.1 Cross-Validation: GridSearchCV

One way to check what the best values are for the hyperparameters would be to manually check the performance of the models with different hyperparameters, but this would be quite inefficient because we may not have time to explore many combinations.

A more efficient technique exploits cross-validation: the training set is split into complementary subsets or "folds." For each fold, the model is trained against the remaining folds using different hyperparameters, where the fold is used as a validation set. This is called *k-fold cross-validation*. Once the model type and hyperparameters have been selected, a final model is trained using these hyperparameters on the full training set, and the generalized error is measured on the test set.

Scikit-Learn's GridSearchCV will do the fine-tuning and the cross validation. All you need to do is tell it which hyperparameters you want it to experiment with and what values to try out and it will evaluate all the possible combinations of hyperparameter values, using cross-validation.

2.2 Other Tuners

The grid search approach is fine when you are exploring relatively few combinations, but when the hyperparameter search space is large, it is often preferable to use *Randomized*

Search through RandomizedSearchCV instead. This class can be used in much the same way as the GridSearchCV class, but instead of trying out all possible combinations, it evaluates a given number of random combinations of hyperparameters by selecting a random value for each hyperparameter at every iteration. An advantage over grid search is that it will explore different values of the hyperparameters at each run, instead of combinations of values set in advance.

Another way to fine-tune your system is to try to combine the models that perform best. If you aggregate the predictions of a group of predictors (such as different regression models), you will often get better predictions than with the best individual predictor. The *Ensemble Methods* allows us to combine several predictors from different trained models to reach an even better predictor.

3. Performance with Hyperparameter Tuning

The code below performs a grid search cross-validation exercise for our Elastic Net Regression to predict stock returns. The k -fold algorithm randomly splits the training set into 10 different subsamples or "folds", the parameter "n_splits" below. Then, each fold is used as a validation set where the model is trained using the other folds. Then, we have k trained models and we can compute the performance of each fold estimation across them in the test set.

Because the k folds are split randomly, a small value of k can lead to noisy results. This means that each time the procedure is run, a different split of the dataset into k -folds can be implemented, and in turn, the distribution of performance scores can be different, resulting in a different mean estimate of model performance. To solve this, we can repeat the k -fold cross-validation process multiple times, the parameter "n_repeat" below, and report the mean performance across all folds and all repeats. This approach is generally referred to as repeated k -fold cross-validation.

Note: The parameter "random_state" makes sure that the algorithm yields the same reproducible output across multiple function calls.

Once we set up the cross validation technique, we evaluate the model for a pre-determined grid of our hyperparameters in the Elastic Net Regression, the overall degree of regularization, α , and the relative importance of Lasso over Ridge regularization, ρ . We rank models across each repeated k -fold validation using their MSE.

```
In [ ]: from sklearn.model_selection import GridSearchCV, RepeatedKFold

model = ElasticNet()
# define model evaluation method
cv = RepeatedKFold(n_splits=10, n_repeats=3, random_state=1)
# define grid
grid = dict()
grid["alpha"] = [1e-5, 1e-4, 1e-3, 1e-2, 1e-1, 0.0, 1.0, 10.0, 100.0]
grid["l1_ratio"] = [0, 0.01, 0.1, 0.2, 0.5, 0.7, 1]
search = GridSearchCV(model, grid, scoring="neg_mean_squared_error", cv=cv, n_jc
```

```
# perform the search
results = search.fit(X_train, y_train)
# summarize
print("MSE: %.3f" % results.best_score_)
print("Config: %s" % results.best_params_)
```

It turns out that the best model from cross-validation is very close to plain linear regression! The model includes a small degree of ridge regularization and no Lasso regularization.

```
In [ ]: # Train the model
e_net = ElasticNet(alpha=0.001, l1_ratio=0) # Using the above hyperparameters
e_net.fit(X_train, y_train)

# calculate the prediction and mean square error
y_pred_elastic = e_net.predict(X_test)
mean_squared_error = np.mean((y_pred_elastic - y_test) ** 2)
print("Mean Squared Error on test set", mean_squared_error)
```

4. Conclusion

In this lesson, we have described the main methods for regularization and fine-tuning techniques to obtain the best combinations of hyperparameters in the linear regression model.

In the next lesson, we will analyze the main features of classification problems in machine learning.

Copyright 2025 WorldQuant University. This content is licensed solely for personal use. Redistribution or publication of this material is strictly prohibited.